

WEB APPLICATION DEVELOPMENT

LAB 9 - HOMEWORK

EXERCISE 6: PASSWORD MANAGEMENT

6.1: Change Password Endpoint

Create DTO:

```
public class ChangePasswordDTO {  
  
    @NotBlank(message = "Current password must not be blank")  
    private String currentPassword;  
  
    @NotBlank(message = "New password must not be blank")  
    @Size(min = 6, message = "New password must be at least 6 characters long")  
    private String newPassword;  
  
    @NotBlank(message = "Confirm password must not be blank")  
    private String confirmPassword;  
}
```

Add to AuthController:

```
@PostMapping("/change-password")  
public ResponseEntity<Map<String, String>> changePassword(  
    @Valid @RequestBody ChangePasswordDTO dto) {  
  
    // 1. Get current user from SecurityContext  
    Authentication authentication =  
        SecurityContextHolder.getContext().getAuthentication();  
    String username = authentication.getName();  
  
    // 2. Verify current password  
    // 3. Check new password matches confirm  
    // 4. Hash and update password  
    // (Delegated to service layer)  
    userService.changePassword(username, dto);  
  
    // 5. Return success message  
    Map<String, String> response = new HashMap<>();  
    response.put(key: "message", value: "Password changed successfully");  
  
    return ResponseEntity.ok(response);  
}
```

Add to UserService:

```
void changePassword(String username, ChangePasswordDTO dto);
```

Add to UserServiceImpl:

```
@Override  
public void changePassword(String username, ChangePasswordDTO dto) {  
    // 1. Load user from database  
    User user = userRepository.findByUsername(username)  
        .orElseThrow(() -> new RuntimeException(message: "User not found"));  
  
    // 2. Verify current password  
    if (!passwordEncoder.matches(dto.getCurrentPassword(), user.getPassword())) {  
        throw new RuntimeException(message: "Current password is incorrect");  
    }  
  
    // 3. Check new password confirmation  
    if (!dto.getNewPassword().equals(dto.getConfirmPassword())) {  
        throw new RuntimeException(message: "New password and confirm password do not match");  
    }  
  
    // 4. Hash and update password  
    String encodedPassword = passwordEncoder.encode(dto.getNewPassword());  
    user.setPassword(encodedPassword);  
  
    // 5. Save updated password  
    userRepository.save(user);  
}
```

Output: /api/auth/change-password (with admin):

Secure_Customer_API / PUT change password

PUT `{{baseURL}}/api/auth/change-password` Send

Docs Params Authorization Headers (10) Body Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON Schema Beautify

```
1 {
2   "currentPassword": "password123",
3   "newPassword": "password456",
4   "confirmPassword": "password456"
5 }
```

Body Cookies Headers (14) Test Results 200 OK • 212 ms • 466 B Save Response

{ } JSON Preview Visualize

```
1 {
2   "message": "Password changed successfully"
3 }
```

Explanation of the code flow of changing password:

1. Client → PUT /api/auth/change-password
 - The client sends an authenticated HTTP PUT request containing currentPassword, newPassword, confirmPassword.
 - The request must include a valid **JWT access token** in the Authorization header. This ensures only logged-in users can change their password.
2. Server → Get current authenticated user
 - Spring Security retrieves the currently authenticated user from the SecurityContext.
 - The JWT token was already validated by JwtAuthenticationFilter.
 - The username stored inside the token is extracted automatically.
3. Server → Verify current password: The server loads the user from the database using the username and verifies the provided currentPassword.
4. Server → Check new password matches confirm password
5. Server → Hash new password using BCrypt
6. Server → Update user password in database
7. Server → Return success response

6.2: Forgot Password

Create password reset token system:

Add fields to User entity:

```
@Column(name = "reset_token")
private String resetToken;

@Column(name = "reset_token_expiry")
private LocalDateTime resetTokenExpiry;
```

Add ForgotPassword DTO:

```
public class ForgotPasswordDTO {
    @NotBlank
    @Email
    private String email;
    public ForgotPasswordDTO(){}
    public ForgotPasswordDTO (String email) {
        this.email = email;
    }
}
```

Add ResetPassword DTO:

```

public class ResetPasswordDTO {
    @NotBlank
    private String token;

    @NotBlank
    @Size(min = 6)
    private String newPassword;

    @NotBlank
    private String confirmPassword;
}

```

Add to Service:

```

    void forgotPassword(ForgotPasswordDTO dto);

    void resetPassword(ResetPasswordDTO dto);

@Override
public void forgotPassword(ForgotPasswordDTO dto) {
    User user = userRepository.findByEmail(dto.getEmail())
        .orElseThrow(() -> new ResourceNotFoundException(message: "Email not found"));

    // 1. Generate reset token
    String token = UUID.randomUUID().toString();

    // 2. Save token + expiry (1 hour)
    user.setResetToken(token);
    user.setResetTokenExpiry(LocalDate.now().plusHours(hours: 1));

    userRepository.save(user);

    // 3. Return token (real app → send via email)
    System.out.println("Password reset token: " + token);
}

```

Forgot password in UserServiceImpl.java

```

@Override
public void resetPassword(ResetPasswordDTO dto) {
    User user = userRepository.findByResetToken(dto.getToken())
        .orElseThrow(() -> new RuntimeException(message: "Invalid reset token"));

    // 1. Check expiry
    if (user.getResetTokenExpiry().isBefore(LocalDate.now())) {
        throw new RuntimeException(message: "Reset token has expired");
    }

    // 2. Confirm password
    if (!dto.getNewPassword().equals(dto.getConfirmPassword())) {
        throw new RuntimeException(message: "Passwords do not match");
    }

    // 3. Hash and update password
    user.setPassword(passwordEncoder.encode(dto.getNewPassword()));

    // 4. Clear token
    user.setResetToken(resetToken: null);
    user.setResetTokenExpiry(resetTokenExpiry: null);

    userRepository.save(user);
}

```

Reset password in UserServiceImpl.java

Add to AuthController:

```
@PostMapping("/forgot-password")
public ResponseEntity<?> forgotPassword(
    @Valid @RequestBody ForgotPasswordDTO dto) {

    userService.forgotPassword(dto);

    return ResponseEntity.ok(
        Map.of(k1: "message", v1: "Password reset token generated (check email)"));
}

@PostMapping("/reset-password")
public ResponseEntity<?> resetPassword(
    @Valid @RequestBody ResetPasswordDTO dto) {

    userService.resetPassword(dto);

    return ResponseEntity.ok(
        Map.of(k1: "message", v1: "Password reset successfully"));
}
```

Output: POST /api/auth/forgot-password

- Generate reset token
- Save token and expiry (e.g., 1 hour)
- Return token (in real app, send via email)

Secure_Customer_API / POST forgot password

POST {{baseURL}} /api/auth/forgot-password

Docs Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

1 {
2 | "email": "john@example.com"
3 }

Body Cookies Headers (14) Test Results 200 OK • 90 ms • 481 B Save Response

{ JSON Preview Visualize

1 {
2 | "message": "Password reset token generated (check email)"
3 }

Forgot password with user = “john”

Output: POST /api/auth/reset-password

- Verify reset token is valid and not expired
- Update password
- Clear reset token

Secure_Customer_API / POST reset password

POST {{baseURL}} /api/auth/reset-password

Docs Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

1 {
2 | "token": "d348054d-b982-4172-8c5b-a6de492361e0",
3 | "newPassword": "password123",
4 | "confirmPassword": "password123"
5 }

Body Cookies Headers (14) Test Results 200 OK • 273 ms • 464 B Save Response

{ JSON Preview Visualize

1 {
2 | "message": "Password reset successfully"
3 }

Reset password with user = “john”

Explanation of the code flow of forgot password:

1. Client → POST /api/auth/forgot-password {email}: The client sends a POST request containing the user's registered email address. This endpoint is **public**, so no authentication is required.
2. Server → Verify email exists: The server checks whether the email exists in the database. If the email is not found, an error response is returned. This prevents generating tokens for non-existent users.
3. Server → Generate reset token: If the email exists, the server generates a **secure random reset token** (UUID or similar). This token is used to verify password reset requests.
4. Server → Save token + expiry (1 hour): This limits how long the token is valid, improving security.
5. Server → Return token (email in real app)

Explanation of the code flow of reset password:

1. Client → POST /api/auth/reset-password: The client submits resetToken, newPassword, confirmPassword. This endpoint is public but protected by the reset token.
2. Server → Verify reset token exists: The server searches for a user with the provided reset token. If the token does not exist, the request is rejected. Preventing unauthorized password resets.
3. Server → Check token expiry: The server checks whether the reset token has expired. Expired tokens are rejected. This protects against replay attacks.
4. Server → Hash new password
5. Server → Clear reset token: After a successful password reset, the server clears resetToken, and resetTokenExpiry. This ensures the token cannot be used.
6. Server → Save updated password: The hashed password and cleared token fields are saved to the database. The user can now log in using the new password.

EXERCISE 7: USER PROFILE MANAGEMENT

7.1: View Profile

Add to UserController.java:

```
@GetMapping("/profile")
public ResponseEntity<UserResponseDTO> getProfile() {

    // 1. Get current authenticated user
    Authentication authentication =
        SecurityContextHolder.getContext().getAuthentication();

    String username = authentication.getName();

    // 2. Fetch user details
    UserResponseDTO user = userService.getCurrentUser(username);

    // 3. Return profile data
    return ResponseEntity.ok(user);
}
```

Output: GET /api/auth/users/profile

Secure_Customer_API / GET view profile

GET {{baseURL}} /api/users/profile

Auth Type: Bearer Token

Token: [REDACTED]

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Body: Cookies Headers (14) Test Results 200 OK • 239 ms • 567 B

```

1 {
2   "id": 1,
3   "username": "admin",
4   "email": "admin@example.com",
5   "fullName": "Admin User",
6   "role": "ADMIN",
7   "isActive": true,
8   "createdAt": "2025-12-09T16:52:59"
9 }

```

View profile with user = “admin”

Explanation of the code flow of the view profile:

1. Client → GET /api/auth/users/profile: The client sends an authenticated GET request to retrieve the current user’s profile information. The request must include a valid JWT token in the header. This ensures only logged-in users can access their profile.
2. Client → Sends JWT in Authorization header: The JWT contains the user’s identity (username) and was issued during login. No request body is required because the server identifies the user from the token.
3. JwtAuthenticationFilter validates token: Before reaching the controller, the request is intercepted by JwtAuthenticationFilter. The filter extracts the JWT from the Authorization header, verifies the token’s signature, checks whether the token is expired. If validation fails, the request is rejected with 401 Unauthorized.
4. Server → Extract username from JWT: Once validated, the filter extracts the username claim from the JWT. This username uniquely identifies the authenticated user.
5. Server → Store authentication in SecurityContext: Spring Security stores the authenticated user inside the SecurityContext. This allows the user identity to be accessed anywhere during request processing.
6. Controller reads username from SecurityContext: The controller retrieves the current authentication object. This avoids passing sensitive data (like username) from the client.
7. Service loads user from database: The controller calls the service layer, which fetches the user from the database using the username. If the user does not exist, an error is returned.
8. Server returns UserResponseDTO: The user entity is converted into a UserResponseDTO and returned to the client. Sensitive fields such as passwords are never exposed.

7.2: Update Profile

Create DTO:

```

public class UpdateProfileDTO {

    @NotBlank(message = "Full name is required")
    private String fullName;

    @Email(message = "Email must be valid")
    private String email;

    // Constructor
    public UpdateProfileDTO() {
    }
}

```

Add endpoint:

```
@PostMapping("/profile/update")
public ResponseEntity<UserResponseDTO> updateProfile(
    @Valid @RequestBody UpdateProfileDTO dto) {

    // 1. Get authenticated user
    Authentication authentication =
        SecurityContextHolder.getContext().getAuthentication();

    String username = authentication.getName();

    // 2. Update profile
    UserResponseDTO updatedUser =
        userService.updateProfile(username, dto);

    // 3. Return updated user
    return ResponseEntity.ok(updatedUser);
}
```

Add to Service:

```
UserResponseDTO updateProfile(String username, UpdateProfileDTO dto);

@Override
public UserResponseDTO updateProfile(String username, UpdateProfileDTO dto) {

    // 1. Load current user
    User user = userRepository.findByUsername(username)
        .orElseThrow(() -> new ResourceNotFoundException(message: "User not found"));

    // 2. Update fields
    user.setFullName(dto.getFullName());

    if (dto.getEmail() != null && !dto.getEmail().isBlank()) {
        user.setEmail(dto.getEmail());
    }

    // 3. Save changes
    User updatedUser = userRepository.save(user);

    // 4. Return updated profile
    return convertToDTO(updatedUser);
}
```

Output: PUT /api/auth/users/profile/update (with john)

The screenshot shows a REST client interface with the following details:

- URL:** `PUT {{baseURL}} /api/users/profile/update`
- Method:** PUT
- Body (raw):**

```
{
  "fullName": "John Smith",
  "email": "john@example.com"
}
```
- Response:** 200 OK, 96 ms, 564 B. The response body is a JSON object:

```
{
  "id": 2,
  "username": "john",
  "email": "john@example.com",
  "fullName": "John Smith",
  "role": "USER",
  "isActive": true,
  "createdAt": "2025-12-09T16:52:59"
}
```


Explanation of the code flow of the updating profile:

1. Client → PUT /api/auth/users/profile/update: The client sends an authenticated PUT request to update profile information. This request must include a valid JWT token.
2. Client → Sends JWT token in authorization header: The client includes the JWT access token in the request header. This ensures only authenticated users can update their own profile.
3. JWT filter validates token: Before the request reaches the controller, the JwtAuthenticationFilter extracts the token from the header, verifies token signature, checks token expiration. Invalid or expired tokens result in 401 Unauthorized.
4. Server → Extract username from SecurityContext: After successful validation, Spring Security stores authentication data in the SecurityContext. The username identifies the user to be updated.
5. Server loads user from database: The service layer retrieves the user entity using the username. If the user does not exist, the request is rejected.
6. Server updates full name & email
7. Server saves updated user: The updated user entity persisted to the database. This operation is performed inside a transactional context to ensure data integrity.
8. Server returns updated profile

7.3: Delete Account

Add to AuthController.java:

```
@DeleteMapping("/account")
public ResponseEntity<?> deleteAccount(@RequestParam String password) {

    // 1. Get authenticated user
    Authentication authentication =
        SecurityContextHolder.getContext().getAuthentication();

    String username = authentication.getName();

    // 2. Delete account (soft delete)
    userService.deleteAccount(username, password);

    // 3. Return response
    return ResponseEntity.ok(
        Map.of(k1: "message", v1: "Account deleted successfully"));
}
```

Add to Service:

```
void deleteAccount(String username, String password);

@Override
public void deleteAccount(String username, String password) {

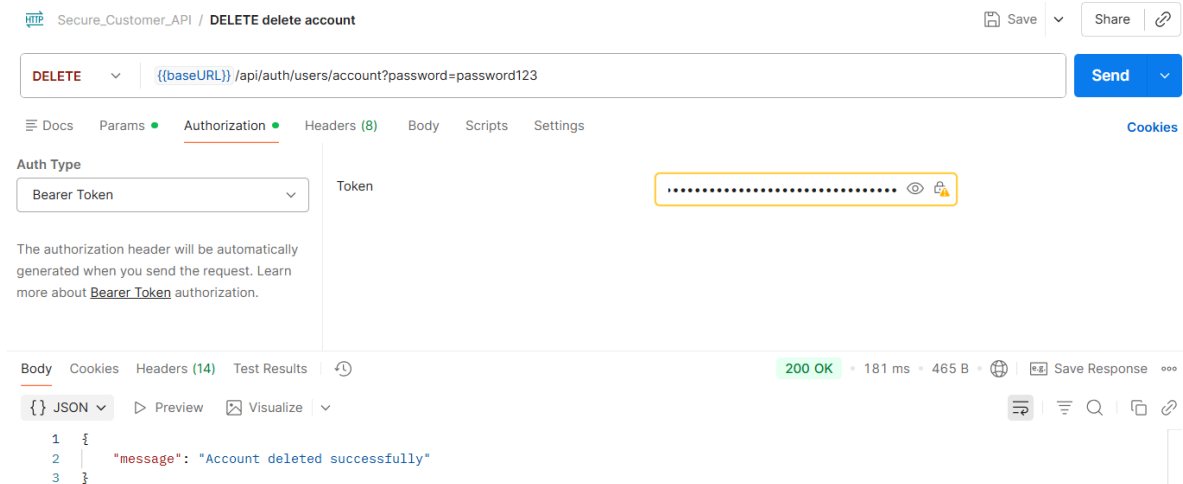
    // 1. Load user
    User user = userRepository.findByUsername(username)
        .orElseThrow(() -> new ResourceNotFoundException(message: "User not found"));

    // 2. Verify password
    if (!passwordEncoder.matches(password, user.getPassword())) {
        throw new RuntimeException(message: "Invalid password");
    }

    // 3. Soft delete
    user.setIsActive(false);

    userRepository.save(user);
}
```


Output: DELETE /api/auth/users/account?password=password123 (with testuser)



Explanation of the code flow of the deleting account:

1. Client → DELETE /api/auth/users/account?password=xxx: The client sends an authenticated DELETE request to delete their own account. The request includes the user's password as a request parameter (for verification), a valid JWT access token. This endpoint performs a soft delete, not a physical deletion from the database.
2. Client → Sends JWT token: The client includes the JWT in the request header. This ensures the request is made by a logged-in user.
3. JWT filter authenticates user: The JwtAuthenticationFilter intercepts the request and extracts the JWT from the header, validates the token signature, checks token expiration. If the token is invalid or expired, the request is rejected with 401 Unauthorized.
4. Server → Retrieve username from SecurityContext: After successful authentication, Spring Security stores the user identity in the SecurityContext. This username identifies which account will be soft deleted.
5. Server verifies password using Bcrypt
6. Server sets isActive = false
7. Server saves user
8. Server returns success message

EXERCISE 8: ADMIN ENDPOINTS

8.1: List All Users

Add to AdminController.java:

```
@GetMapping("/users")
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<List<UserResponseDTO>> getAllUsers() {

    List<UserResponseDTO> users = userService.getAllUsers();

    return ResponseEntity.ok(users);
}
```

Add to Service:

```
List<UserResponseDTO> getAllUsers();

@Override
public List<UserResponseDTO> getAllUsers() {

    return userRepository.findAll()
        .stream()
        .map(this::convertToDTO)
        .toList();
}
```

Output:

Secure_Customer_API / GET get all users with admin

GET {{baseUrl}} /api/admin/users

Auth Type: Bearer Token

Token: [REDACTED]

200 OK • 220 ms • 999 B

```
1 [
2   {
3     "id": 1,
4     "username": "admin",
5     "email": "admin@example.com",
6     "fullName": "Admin User",
7     "role": "ADMIN",
8     "isActive": true,
9     "createdAt": "2025-12-09T16:52:59"
10  },
11  {
12    "id": 2,
13    "username": "john",
14    "email": "john@example.com",
15    "fullName": "John Smith",
16    "role": "USER",
17    "isActive": true,
18    "createdAt": "2025-12-09T16:52:59"
19  }
20 ]
```

Explanation of the code flow of the view all users with administrative permission:

1. Client → GET /api/admin/users: The admin client sends a GET request to retrieve the list of all users in the system. This endpoint is restricted to **ADMIN users only**.
2. Client → Sends JWT token: The client includes a valid JWT access token in the request header. The token contains the authenticated user's identity and role.
3. JwtAuthenticationFilter validates token: Before reaching the controller, the request is intercepted by JwtAuthenticationFilter. The filter extracts the JWT from the header, verifies the token signature, checks token expiration.
4. Spring Security → Check role (ADMIN): Spring Security evaluates the authorization rule. If the user does not have ROLE_ADMIN, access is denied. The server responds with 403 Forbidden. This ensures only administrators can access this endpoint.
5. Service loads all users: After authorization, the controller calls the service layer to retrieve all users from the database. Both active and inactive users are retrieved, depending on implementation.
6. Entity → DTO mapping: Each User entity is converted into a UserResponseDTO, which prevents exposure of sensitive fields (password, tokens), and ensures clean API responses.
7. Server returns user list: The server returns a list of user DTOs.

8.2: Update User role

Create DTO:

```
public class UpdateRoleDTO {

    @NotNull(message = "Role is required")
    private Role role;
```

Add endpoint:

```

@PutMapping("/users/{id}/role")
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<UserResponseDTO> updateUserRole(
    @PathVariable Long id,
    @Valid @RequestBody UpdateRoleDTO dto) {

    UserResponseDTO updatedUser =
        userService.updateUserRole(id, dto.getRole());

    return ResponseEntity.ok(updatedUser);
}

```

Add to Service:

```

    UserResponseDTO updateUserRole(Long userId, Role role);

@Override
public UserResponseDTO updateUserRole(Long userId, Role role) {

    // 1. Load user
    User user = userRepository.findById(userId)
        .orElseThrow(() -> new ResourceNotFoundException(message: "User not found"));

    // 2. Update role
    user.setRole(role);

    // 3. Save changes
    User updatedUser = userRepository.save(user);

    // 4. Return updated user
    return convertToDTO(updatedUser);
}

```

Output:

The screenshot shows a REST client interface for a PUT request to the endpoint `PUT {{baseURL}}/api/admin/users/2/role`. The request body is a JSON object: `{ "role": "ADMIN" }`. The response is a 200 OK status with a response time of 367 ms and a body size of 565 B. The response body is a JSON object containing user details: `{ "id": 2, "username": "john", "email": "john@example.com", "fullName": "John Smith", "role": "ADMIN", "isActive": true, "createdAt": "2025-12-09T16:52:59" }`.

Change from user to administrator

Secure_Customer_API / PUT update role

PUT `{{baseUrl}} /api/admin/users/2/role` Send

Docs Params Authorization Headers (10) Body Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL ☒ JSON Schema Beautify

```

1 {
2   "role": "USER"
3 }

```

Body Cookies Headers (14) Test Results 200 OK • 116 ms • 564 B Save Response

{} JSON Preview Visualize

```

1 {
2   "id": 2,
3   "username": "john",
4   "email": "john@example.com",
5   "fullName": "John Smith",
6   "role": "USER",
7   "isActive": true,
8   "createdAt": "2025-12-09T16:52:59"
9 }

```

Change from administrator to user

Explanation of the code flow of updating user role:

1. Admin → PUT /api/admin/users/{id}/role: An administrator sends a PUT request to update the role of a specific user. The request includes id of the target user (path variable), request body containing the new role.
2. Admin → Sends JWT token: The admin includes a valid JWT access token in the request header. The token identifies the requester and contains their assigned role.
3. JWT filter validates token: The request is intercepted by JwtAuthenticationFilter, which extracts the JWT from the header, validates the token signature, checks token expiration. Invalid tokens result in 401 Unauthorized
4. @PreAuthorize checks ADMIN role: Spring Security enforces role-based authorization using @PreAuthorize("hasRole('ADMIN')"). Only user with ROLE_ADMIN are allowed. Other users receive 403 Forbidden.
5. Service loads user by ID: After authorization, the service layer retrieves the user entity from the database using the provided ID. If the user does not exist, an error is returned.
6. Service updates role: The user's role is updated based on the request body. This immediately changes the user's permissions.
7. Server saves user
8. Server returns updated user

8.3: Deactivate/Activate User

Add to AdminController.java:

```

@PatchMapping("/users/{id}/status")
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<UserResponseDTO> toggleUserStatus(
    @PathVariable Long id) {

    UserResponseDTO updatedUser =
        userService.toggleUserStatus(id);

    return ResponseEntity.ok(updatedUser);
}

```

Add to Service:

```

UserResponseDTO toggleUserStatus(Long userId);

```

```

@Override
public UserResponseDTO toggleUserStatus(Long userId) {

    // 1. Load user
    User user = userRepository.findById(userId)
        .orElseThrow(() -> new ResourceNotFoundException(message: "User not found"));

    // 2. Toggle isActive
    user.setIsActive(!user.getIsActive());

    // 3. Save changes
    User updatedUser = userRepository.save(user);

    // 4. Return updated user
    return convertToDTO(updatedUser);
}

```

Output:

Secure_Customer_API / PATCH change status by admin

PATCH Send

Auth Type: Bearer Token

Token:

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Body: JSON Preview Visualize

```

1 {
2   "id": 2,
3   "username": "john",
4   "email": "john@example.com",
5   "fullName": "John Smith",
6   "role": "USER",
7   "isActive": false,
8   "createdAt": "2025-12-09T16:52:59"
9 }

```

200 OK • 85 ms • 565 B

Change the status of user id = 2 from active to inactive

Secure_Customer_API / PATCH change status by admin

PATCH Send

Auth Type: Bearer Token

Token:

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Body: JSON Preview Visualize

```

1 {
2   "id": 2,
3   "username": "john",
4   "email": "john@example.com",
5   "fullName": "John Smith",
6   "role": "USER",
7   "isActive": true,
8   "createdAt": "2025-12-09T16:52:59"
9 }

```

200 OK • 30 ms • 564 B

Change the status of user id = 2 from inactive to active

Explanation of the code flow of the deactivating/activating user

1. Admin → PATCH /api/admin/users/{id}/status: An administrator sends a PATCH request to toggle the activation status of a user account. The {id} identifies the target user. The operation switches the user between active and inactive.
2. Admin → Sends JWT token: The admin includes a valid JWT access token in the request header. This token identifies the requester and contains their assigned role.
3. JWT filter validates token: Before reaching the controller, the request is intercepted by JwtAuthenticationFilter, which extracts the JWT from the header, validates the token signature, checks token expiration. Invalid or expired tokens result in 401 Unauthorized.
4. @PreAuthorize checks ADMIN role: Spring Security enforces role-based access control using @PreAuthorize("hasRole('ADMIN')"). Only users with ROLE_ADMIN are authorized. Non-admin users receive 403 Forbidden.
5. Service loads user by ID: After authorization, the service layer retrieves the user entity from the database using the provided ID. If the user does not exist, an error response is returned.
6. Service toggles isActive: The service toggles the user's activation status. If isActive value is from true to false, which means that user is deactivated. If isActive value is from false to true, which means that user is activated.
7. Server saves user
8. Server returns updated user

EXERCISE 9: REFRESH TOKEN

9.1: Create Refresh Token Entity

```
public class RefreshToken {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    // Many refresh tokens can belong to one user  
    @ManyToOne(fetch = FetchType.LAZY)  
    @JoinColumn(name = "user_id", nullable = false)  
    private User user;  
  
    @Column(unique = true, nullable = false)  
    private String token;  
  
    @Column(nullable = false)  
    private LocalDateTime expiryDate;  
}
```

9.2: Generate Refresh Token

Update LoginResponseDTO.java:

```
public class LoginResponseDTO {  
  
    private String accessToken;  
    private String refreshToken;  
    private String tokenType = "Bearer";  
    private String username;  
    private String email;  
    private String role;  
}
```

Create RefreshTokenRepository.java:

```

@Repository
public interface RefreshTokenRepository extends JpaRepository<RefreshToken, Long> {

    Optional<RefreshToken> findByToken(String token);

    void deleteById(Long userId);
}

```

Refresh token creation logic:

```

@Autowired
private RefreshTokenRepository refreshTokenRepository;

private RefreshToken createRefreshToken(User user) {

    // Optional: delete old refresh tokens (one active session)
    refreshTokenRepository.deleteByUserId(user.getId());

    RefreshToken refreshToken = new RefreshToken();
    refreshToken.setUser(user);
    refreshToken.setToken(UUID.randomUUID().toString());
    refreshToken.setExpiryDate(LocalDate.now().plusDays(days: 7));

    return refreshTokenRepository.save(refreshToken);
}

```

Update login to return refresh token:

```

@Override
public LoginResponseDTO login(LoginRequestDTO loginRequest) {

    // 1. Authenticate user
    Authentication authentication = authenticationManager.authenticate(
        new UsernamePasswordAuthenticationToken(
            loginRequest.getUsername(),
            loginRequest.getPassword()
        )
    );

    SecurityContextHolder.getContext().setAuthentication(authentication);

    // 2. Generate access token (JWT)
    String accessToken = tokenProvider.generateToken(authentication);

    // 3. Load user
    User user = userRepository.findByUsername(loginRequest.getUsername())
        .orElseThrow(() -> new ResourceNotFoundException(message: "User not found"));

    // Optional: block deactivated users
    if (!user.getIsActive()) {
        throw new RuntimeException(message: "Account is deactivated");
    }

    // 4. Generate refresh token
    RefreshToken refreshToken = createRefreshToken(user);

    // 5. Return both tokens
    return new LoginResponseDTO(
        accessToken,
        refreshToken.getToken(),
        user.getUsername(),
        user.getEmail(),
        user.getRole().name()
    );
}

```


Output:

Secure_Customer_API / POST login

POST {{baseURL}} /api/auth/login

Body

```
1 {
2   "username": "john",
3   "password": "password123"
4 }
```

200 OK • 511 ms • 747 B

JSON

```
1 {
2   "accessToken": "eyJhbGciOiJIUzUxMiJ9.eyJzdWIiOiJqb2huIiwiaWF0IjoxNzY1NjMwMDk3LCJleHAiOjE3NjU3MTY0OTd9.
3     APgvicnP_JuAgZfu2NhisE-aLQDnvCWItUwZMGkWXD8COHJ_CdGKlhZhuCwdFYJjrTga6f0UYSE3IffYIVFTzg",
4   "refreshToken": "cde4d2aa-1b05-4e35-b081-f0840697ccf4",
5   "tokenType": "Bearer",
6   "username": "john",
7   "email": "john@example.com",
8   "role": "USER"
9 }
```

Explanation of the code flow of generating refresh tokens:

1. Client → POST /api/auth/login: The client sends a POST request to authenticate using {"username": "user", "password": "password"}. This is a public endpoint.
2. Server → Authenticate user: Spring Security authenticates the user using the AuthenticationManager. Credentials are verified against stored hashed passwords. Invalid credentials result in 401 Unauthorized.
3. Server → Generate JWT access token: After successful authentication, the server generates a **short-lived JWT access token**.
 - Contains username and role.
 - Used to access protected endpoints.
 - Valid for a short duration (e.g., minutes or hours).
4. Server → Generate refresh token (7 days): The server generates a **long-lived refresh token**.
 - Random, unique value (UUID)
 - Expiry set to **7 days**
 - Used only to obtain new access tokens.
5. Server → Save refresh token in database: The refresh token is stored in the database and linked to the user.
 - Enables token validation.
 - Allows token revocation.
 - Supports token rotation.
6. Server → Return access + refresh tokens: The server returns both tokens to the client
 - Access token → used for API calls
 - Refresh token → used to request new access tokens

9.3: Refresh Access Token

Create RefreshTokenDTO.java:

```
public class RefreshTokenDTO {

    @NotBlank(message = "Refresh token is required")
    private String refreshToken;
```

Add to JwtTokenProvider.java:

```

// Generate JWT token from username (refresh flow)
public String generateTokenFromUsername(String username) {

    Date now = new Date();
    Date expiryDate = new Date(now.getTime() + jwtExpiration);

    SecretKey key =
        Keys.hmacShaKeyFor(jwtSecret.getBytes(StandardCharsets.UTF_8));

    return Jwts.builder()
        .subject(username)
        .issuedAt(now)
        .expiration(expiryDate)
        .signWith(key)
        .compact();
}

```

Add to Service:

```

        LoginResponseDTO refreshAccessToken(String refreshToken);

@Override
public LoginResponseDTO refreshAccessToken(String refreshToken) {

    // 1. Find refresh token in DB
    RefreshToken token = refreshTokenRepository.findByToken(refreshToken)
        .orElseThrow(() -> new RuntimeException(message: "Invalid refresh token"));

    // 2. Check expiry
    if (token.getExpiryDate().isBefore(LocalDateTime.now())) {
        refreshTokenRepository.delete(token);
        throw new RuntimeException(message: "Refresh token expired");
    }

    // 3. Get user
    User user = token.getUser();

    if (!user.getIsActive()) {
        throw new RuntimeException(message: "User account is deactivated");
    }

    // 4. Generate new access token
    String newAccessToken =
        tokenProvider.generateTokenFromUsername(user.getUsername());

    // 5. Rotate refresh token
    refreshTokenRepository.delete(token);
    RefreshToken newRefreshToken = createRefreshToken(user);

    // 6. Return new access token
    return new LoginResponseDTO(
        newAccessToken,
        newRefreshToken.getToken(),
        user.getUsername(),
        user.getEmail(),
        user.getRole().name()
    );
}

```

Add to AuthController.java:

```
@PostMapping("/refresh")
public ResponseEntity<LoginResponseDTO> refreshToken(
    @Valid @RequestBody RefreshTokenDTO dto) {

    LoginResponseDTO response =
        userService.refreshAccessToken(dto.getRefreshToken());

    return ResponseEntity.ok(response);
}
```

Output:

The screenshot shows a REST client interface with the following details:

- Endpoint:** POST `{{baseURL}}/api/auth/refresh`
- Body:**

```
{
  "refreshToken": "cde4d2aa-1b95-4e35-b081-f8040697ccf4"
}
```
- Response:** 200 OK, 123 ms, 747 B. The response body is a JSON object:

```
{
  "accessToken": "eyJhbGciOiJIUzUxMiJ9.eyJzdWIiOiJqb2huIiwiaWF0IjoxNzY1NjMxODc1LCJleHAiOiJlMjU3MTgyNzV9.69dqlseyNknsZX6aUDRl3W2GZyN-dUPggIXIUecJlJjVlXomhs5GE3fRanuotI-cBmdf0J0h_NjluowjF6kKw",
  "refreshToken": "358e7715-bae4-4473-a7c4-8fcf3477437c",
  "tokenType": "Bearer",
  "username": "john",
  "email": "john@example.com",
  "role": "USER"
}
```

Refresh access token with old refresh token rotated (with john)

Explanation of the code flow of the refreshing access token:

1. Client → POST /api/auth/refresh: The client sends a POST request to request a new access token. This endpoint is public but protected by the refreshing token.
2. Client → Sends refresh token: The refresh token is a long-lived credential issued during login.
 - It is not a JWT
 - It is stored securely on the client's side.
 - Used only to obtain new access tokens.
3. Server → Validate refresh token exists: The server searches for the refresh token in the database. If the token does not exist, the request is rejected. Prevents usage of forged tokens
4. Server → Check refresh token expiry: The server checks whether the refresh token has expired.
 - Expired tokens are rejected
 - Protects against replay attacks
5. Server → Load user: The server retrieves the user associated with the refresh token. This ensures the new access token is issued to the correct user.
6. Server → Generate new access token: A new short-lived JWT access token is generated. The user does **not** need to re-enter credentials.
7. Server → Return new access token